

WORDLEAPP

Ein aesthetisches Wordle-Klon als Terminal User Interface

C# · .NET 8 · Spectre.Console



Überblick

WordleApp ist ein vollständiger Klon des Spiels *Wordle*, gebaut als **Terminal User Interface (TUI)** in C# / .NET 8.

- Läuft komplett in der Konsole — dank **Spectre.Console** sieht es trotzdem poliert aus: Figlet-Banner, abgerundete Panels, farbige Kacheln
- **Zwei Sprachen** (Deutsch/Englisch), beliebig erweiterbar per JSON
- **Drei Schwierigkeitsgrade** mit eigenen Regeln & Punktemultiplikatoren
- **Externe Wortquelle** über eine Zufallswort-API + optionale Wörterbuch-Validierung
- **Persistente Einstellungen & Bestenliste** in JSON-Dateien
- **Adaptives Layout**, das sich live an die Terminalgröße anpasst

Kern-Features

Feature	Beschreibung
 Spielmodi	Easy / Normal / Hard — unterschiedliche Regeln & Punktemultiplikatoren
 Mehrsprachig	Deutsch & Englisch, neue Sprachen ohne Codeänderung
 Bestenliste	Kumulierte Statistik pro Spieler (Siege, Streak, Bestzeit)
 Adaptive UI	Kachelgröße & Layout passen sich live an Fenstergröße an
 Live-Wortquelle	Zufallswort-API + optionale Wörterbuch-Validierung
 Konfigurierbar	Farben, Rundenlänge, API-Endpunkte über <code>config.json</code>

Architektur — Strikte Schichtentrennung

Schicht	Ordner	Zuständigkeit	Konsolen-Zugriff
Models	Models/	Reine Datenstrukturen, keine Logik	Nein
Core	Core/	Spielregeln, Zustandsmaschinen, Menüsteuerung	Nein
Services	Services/	Dateizugriff, HTTP, Übersetzung	Nein
UI	UI/	Einzige Schicht mit <code>Spectre.Console</code>	Ja, exklusiv

"Never mix console output with game logic. All drawing must happen in `FrontendRenderer.cs`." — Projekt-Leitprinzip

Programmablauf (Lifecycle)

```
Program.Main()
├── ConfigService.Load()           → GameConfig
├── LocalizationService(...)
├── ConsoleSession.Start()        → Terminal vorbereiten
├── AppShell.Run()
│   ├── WaitForUsableWindow()
│   ├── WordPool.WarmUp(...)      → Wörter im Hintergrund vorladen
│   ├── DrawLoadingScreen()
│   └── RunMainMenu() ← Schleife
│       ├── Play                 → WordleGame.Run(level)
│       ├── Leaderboard
│       ├── How to Play
│       └── Settings
```

Kein DI-Framework — alle Abhängigkeiten werden von Hand in `Program.cs` verdrahtet.

Die kniffligste Logik: WordEvaluator

Der klassische Wordle-**Doppelbuchstaben-Fall**: Enthält das Zielwort einen Buchstaben nur einmal, der Guess ihn aber zweimal, darf nur **eine** Stelle als "vorhanden" markiert werden.

Lösung: Zwei-Durchlauf-Algorithmus

1. **Pass 1** — alle exakten Treffer (grün) markieren & Zielbuchstaben "verbrauchen"
2. **Pass 2** — für jeden Rest: kommt der Buchstabe *woanders* im (noch nicht verbrauchten) Zielwort vor? → gelb, sonst grau

Zielwort `GHOST`, Guess `SASSY` → nur das **erste** S wird markiert, obwohl `SASSY` drei S enthält.

Punkteberechnung

Drei Komponenten, in dieser Priorität:

1. **Grundpunkte fürs Gewinnen** — 1000 (nur bei Sieg)
2. **Bonus für ungenutzte Versuche** — 150 pro übrigem Versuch
3. **Geschwindigkeitsbonus** — linear von 600 auf 0 über die ersten 120s

Die Summe wird mit dem **Schwierigkeits-Multiplikator** skaliert:

Schwierigkeit	Multiplikator
Easy	0,75×
Normal	1,0×
Hard	1,5×

Eine verlorene Runde ist **immer 0 Punkte**, unabhängig von investierter Zeit.

Externe Anbindung: DictionaryApiService

Zwei HTTP-Aufrufe, behandelt als *unzuverlässig und langsam*:

- **Random-Word-API** — liefert Zielwörter (via `WordPool`, im Hintergrund vorgeladen)
- **Dictionary-API** — validiert Rateversuche

Robustheitsprinzipien:

- **Eigene, enge Timeouts** pro Anfrage (Validierung braucht ein enges Zeitlimit)
- **"Fails open"** — nur ein explizites 404 lehnt einen Guess ab; Timeouts/Fehler lassen ihn durch
- Nach **2 Fehlschlägen** wird Validierung für die Sitzung deaktiviert
- Antworten werden **gecacht**, kein Wort wird zweimal abgefragt

Adaptive Darstellung

`FrontendRenderer.cs` — die **einzigste** Klasse, die `Spectre.Console` direkt nutzt (> 1100 Zeilen).

- **Adaptive Kachelgröße**: 6 Qualitätsstufen, größtmögliche Kachel wird gewählt
- **Live-Neuberechnung** bei Fenstergrößenänderung (Resize-Polling alle 40ms, da .NET kein natives Resize-Event kennt)
- **"Shimmer"-Animation**: graue Welle über die aktive Zeile während der Wörterbuch-Prüfung (`AnsiConsole.Live`, kein Flackern)
- **Fallback ohne Farbunterstützung**: Klammer-Notation `[[W]]` / `(0)` / `x`
- **Markup-Escaping**: Spielernamen werden immer escaped, um Spectre-Markup-Injection zu verhindern

Konfiguration & Robustheit

`config.json` — Sprache, Spielername, Schwierigkeit, Theme-Farben, API-Endpunkte. Von Hand editierbar, Werte werden beim Laden geklemmt (`Clamp()`).

Durchgängiges Prinzip: Nichts darf das Spiel zum Absturz bringen.

Situation	Verhalten
<code>config.json</code> fehlt/kaputt	Standardwerte
API nicht erreichbar	Fehlermeldung bzw. Validierung deaktiviert
Terminal zu klein	Eigener Hinweisbildschirm
Keine Farbunterstützung	Klammer-Notation
Absturz / Strg+C	Cursor & Farben werden trotzdem wiederhergestellt

Bekannte Grenzen

- **Keine automatisierten Tests** (obwohl Schlüsselmethoden `virtual` und damit testbar designt sind)
- **Kein Multiplayer**, keine Online-Bestenliste — rein lokal
- **Kein täglicher Wordle-Modus** — jedes Zielwort ist zufällig
- **Reine Flatfile-Persistenz** (JSON) statt Datenbank — für den Anwendungsfall ausreichend

DANKE

Vollständige technische Dokumentation:

Repository → `docs/` (MkDocs-Seite)

Quick Start:

```
cd WordleApp  
dotnet run
```